DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

TEXTIFY

Console Messaging Application

OOP Project Report

Group Members		
S1	HUZAIFA JAWED	| Roll No: CT-24077
S2	SAIM UZ ZAMAN	| Roll No: CT-24078
S3	MUHAMMAD MOSID KHAN	| Roll No: CT-24100

Built with C++ | OOP Principles | Design Patterns

1. Project Overview

Textify is a console-based messaging application developed in C++ using core Object-Oriented Programming principles. The application simulates a real-world group and personal chat system where users can exchange text messages, images, and video references through a clean terminal interface.

The project demonstrates practical application of OOP concepts including abstraction, inheritance, polymorphism, and encapsulation, along with three classic design patterns: the Factory Pattern, the Observer Pattern, and dynamic polymorphism via virtual dispatch.

2. Features & Capabilities

2.1 Messaging

- Send group messages to all members simultaneously
- Send private personal messages between two users
- Support for three message types: Text, Image, and Video
- Real-time console output with timestamped logs

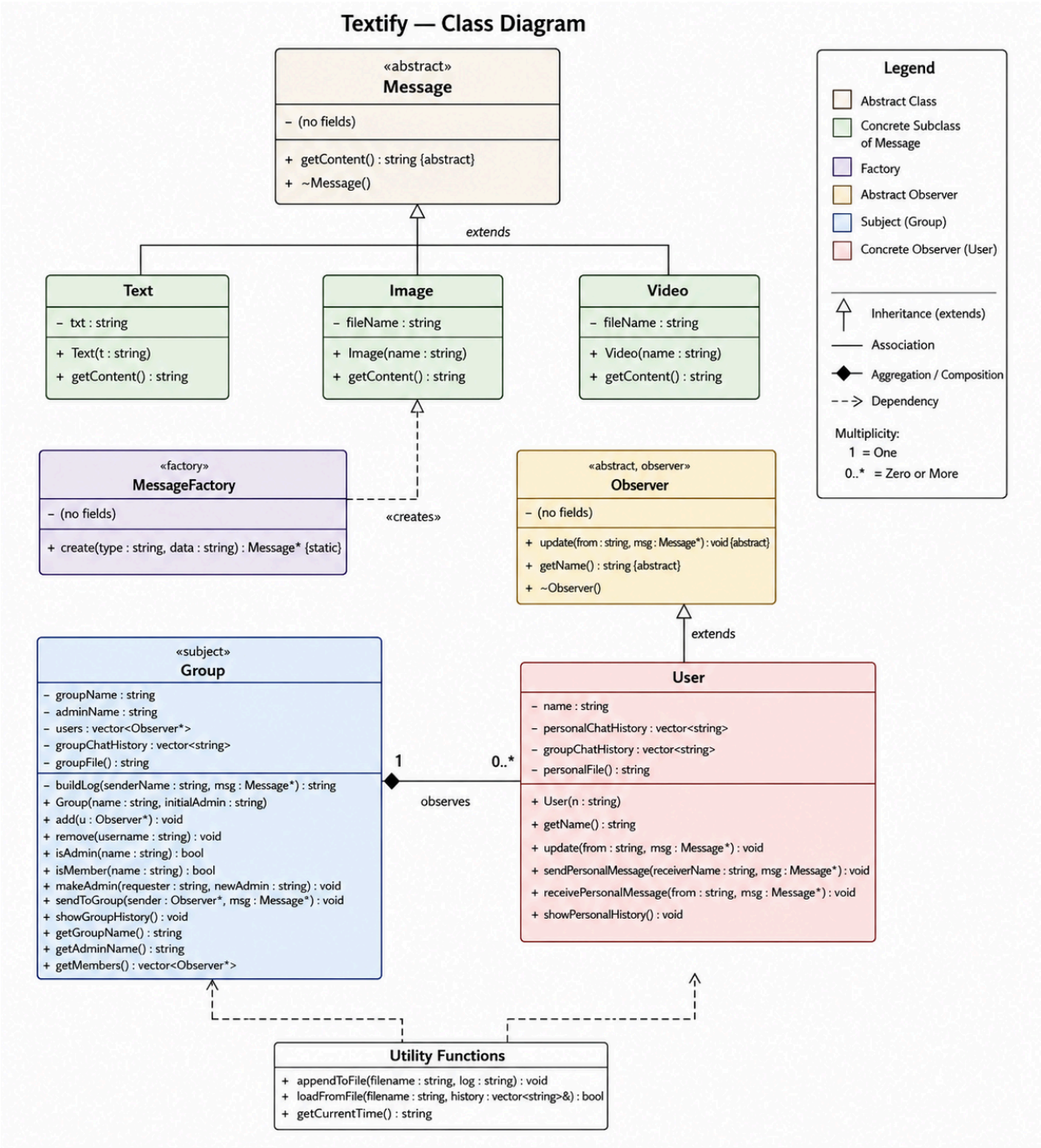
2.2 Group Management

- Admin-controlled member addition and removal
- Admin role transfer to any current group member
- View all current group members with admin indicator
- Protection against removing the current admin

2.3 Chat History & Persistence

- All group messages persisted to a dedicated group .txt file
- Personal messages persisted per-user to individual .txt files
- History automatically reloaded on next application startup
- In-memory history available for both group and personal chats

3. Architecture & Class Design



3.1 Class Hierarchy

The application is structured around a clean hierarchy of abstract and concrete classes. The diagram below illustrates the relationships:

Message (Abstract)	Observer (Interface)	Group (Subject)
Text Image Video	User (implements Observer)	Holds vector<Observer*> Notifies all members

3.2 Class Descriptions

Message (Abstract Base Class)

The Message class defines the interface for all message types through a pure virtual method getContent(). This enforces the contract that every message subclass must be able to produce its content as a string, enabling polymorphic behavior throughout the system.

```
class Message {
public:
    virtual string getContent() const = 0;
    virtual ~Message() {}
};
```

Text, Image, Video (Concrete Message Types)

Three concrete classes extend Message, each overriding getContent() to return a formatted string. Text wraps a plain string, while Image and Video append the appropriate file extension (.jpeg and .mp4 respectively), simulating real media message types.

MessageFactory

Implements the Factory Pattern to decouple message object creation from the rest of the application. The static create() method accepts a type string and data payload, returning the correct polymorphic Message* or nullptr for unknown types.

```
Message* MessageFactory::create(const string& type, const string& data) {
```

```
    if (type == "text")  return new Text(data);
    if (type == "image") return new Image(data);
    if (type == "video") return new Video(data);
    return nullptr;
}
```

Observer (Interface)

Defines the observer interface with two pure virtual methods: update() for receiving group notifications, and getName() for identification. This interface allows the Group class to manage a heterogeneous list of observers without knowing their concrete types.

User (Observer + Messaging)

The User class is the core entity of the system. It implements the Observer interface and adds personal messaging capabilities. Key responsibilities include:

- Maintaining separate in-memory histories for personal and group chats
- Persisting personal messages to a user-specific .txt file on disk
- Loading personal history from disk on construction (persistence across sessions)
- Implementing update() to receive and log group broadcast messages

Group (Subject / Observable)

Group acts as the Subject in the Observer Pattern. It manages a list of Observer pointers, broadcasts messages to all members except the sender, and handles admin-based membership management. It also persists group chat history to a group-specific file, reloading it on startup.

4. Design Patterns

Pattern	Where Applied	Purpose
Factory	`MessageFactory::create()`	Decouples message instantiation from business logic; clients request a type string and receive the correct subclass without knowing constructors.
Observer	`Group (Subject) → User (Observer)`	Enables one-to-many event notification. When a group message is sent, all registered Users are automatically notified without tight coupling.
Polymorphism	`Message* base pointer`	Allows Text, Image, and Video objects to be stored and processed uniformly. The correct

		getContent() is resolved at runtime via virtual dispatch.

5. OOP Principles Applied

5.1 Encapsulation

All class data members are private. The User class hides its chat history vectors and file name derivation logic, exposing only the necessary public interface (send, receive, show history). The Group class similarly hides its member list and admin name, controlling access through its public methods.

5.2 Abstraction

Message and Observer are both abstract classes (containing pure virtual methods) that define contracts without implementations. The rest of the system programs against these abstractions rather than concrete types, making the codebase extensible and loosely coupled.

5.3 Inheritance

Text, Image, and Video each inherit from Message. User inherits from Observer. This creates clear IS-A relationships that allow polymorphic usage throughout the application without conditional type checks.

5.4 Polymorphism

Message pointers are used throughout the system. When getContent() is called on a Message*, the correct derived class method is invoked at runtime. Similarly, Observer* pointers in the Group member list allow the group to call update() without knowing whether the object is a User or any future Observer subclass.

6. File Structure & Persistence

6.1 Generated Files

The application automatically generates and manages the following files at runtime:

File Name	Contents
`TextifyGroup_chat.txt`	All group broadcast messages with timestamps
`Mosid_personal.txt`	Personal message history for user Mosid

`Saim_personal.txt`	Personal message history for user Saim
`Huzaifa_personal.txt`	Personal message history for user Huzaifa
`<NewUser>_personal.txt`	Auto-created for any dynamically added user

6.2 Persistence Logic

Two utility functions handle all file I/O. appendToFile() opens a file in append mode (ios::app) to add a new log line without overwriting existing content. loadFromFile() reads all non-empty lines from a file into a vector<string> on startup. This ensures complete message history survives application restarts.

7. User Interface & Menu System

The application runs an interactive console loop with eight options. Each option is handled by a dedicated function, keeping main() clean and the logic modular:

#	Option	Description
1	**Group Message**	Send a text/image/video message to the entire group
2	**Personal Message**	Send a private message between two specific users
3	**View Group History**	Display all group messages logged in the current session and from previous runs
4	**View Personal History**	Display personal chat history for a specified user
5	**Add / Remove User**	Admin-only: add an existing or new user, or remove a current member
6	**Make Admin**	Admin-only: transfer admin privileges to another group member
7	**View Group Members**	List all current group members, highlighting the admin
8	**Exit**	Gracefully exits the application and frees all allocated memory

8. Sample Session Output

Below is an example console session demonstrating a group message followed by a personal message:

```
==================================================
```

```
                 Welcome to Textify
        Simple Console Messaging Application
==================================================
[Chat history is saved to .txt files and reloaded on startup]

Mosid added to TextifyGroup
Saim added to TextifyGroup
Huzaifa added to TextifyGroup

Select option (1-8): 1
Sender name: Mosid
Message type (text/image/video): text
Content: Hey everyone! Welcome to Textify.
[14:32:01] Mosid to TextifyGroup: Text: Hey everyone! Welcome to Textify.
[14:32:01] Mosid to group: Text: Hey everyone! Welcome to Textify.  <- (Saim
notified)
[14:32:01] Mosid to group: Text: Hey everyone! Welcome to Textify.  <- (Huzaifa
notified)

Select option (1-8): 2
Sender name: Saim
Receiver name: Huzaifa
Message type (text/image/video): image
Content: profile_photo
[14:32:45] Saim to Huzaifa: Image: profile_photo.jpeg
```

9. Memory Management

The application uses raw pointers for polymorphic collections, following classic C++ OOP patterns. Memory management is handled as follows:

- Message objects are created via MessageFactory, used immediately, and deleted after each send operation
- User objects created during runtime (via option 5) are pushed onto the users vector and deleted in the cleanup loop in main()
- The Group class stores Observer* pointers but does not own them — it does not delete them, preventing double-free issues

A future improvement would be to replace raw pointers with std::unique_ptr and std::shared_ptr for fully automatic lifetime management.

10. Limitations & Future Improvements

10.1 Current Limitations

- Single-threaded: no concurrent message delivery simulation
- No user authentication or password protection

- No message deletion or editing capability
- File storage uses plain text; no encryption or binary format
- Username lookup is case-sensitive

10.2 Potential Enhancements

- Replace raw pointers with smart pointers (unique_ptr / shared_ptr)
- Add a Command Pattern for undo/redo of messages
- Implement a Strategy Pattern for different storage backends (file, SQLite, etc.)
- Add unit tests using Google Test or Catch2
- Extend to a networked version using socket programming
- Add message search and filtering functionality

11. Build & Run Instructions

11.1 Requirements

- C++17 compatible compiler (g++ 7+ or clang++ 5+)
- Standard library only — no external dependencies

11.2 Compile & Run

```
# Compile
g++ -std=c++17 -Wall -o textify main.cpp

# Run
./textify

# On Windows (MinGW)
g++ -std=c++17 -Wall -o textify.exe main.cpp
textify.exe
```

12. Summary

Textify successfully demonstrates a clean, modular C++ application built entirely on OOP principles and industry-standard design patterns. The project achieves:

- Clean separation of concerns between message types, users, and the group system
- Extensibility: new message types can be added by subclassing Message and updating the factory
- Persistence: chat history survives application restarts via file-based logging

- Maintainability: dedicated handler functions keep main() concise and each feature independently testable

The codebase serves as a strong foundation for understanding how real-world messaging systems apply OOP — from WhatsApp-style group broadcasts to private direct messaging — all implemented from scratch in standard C++.